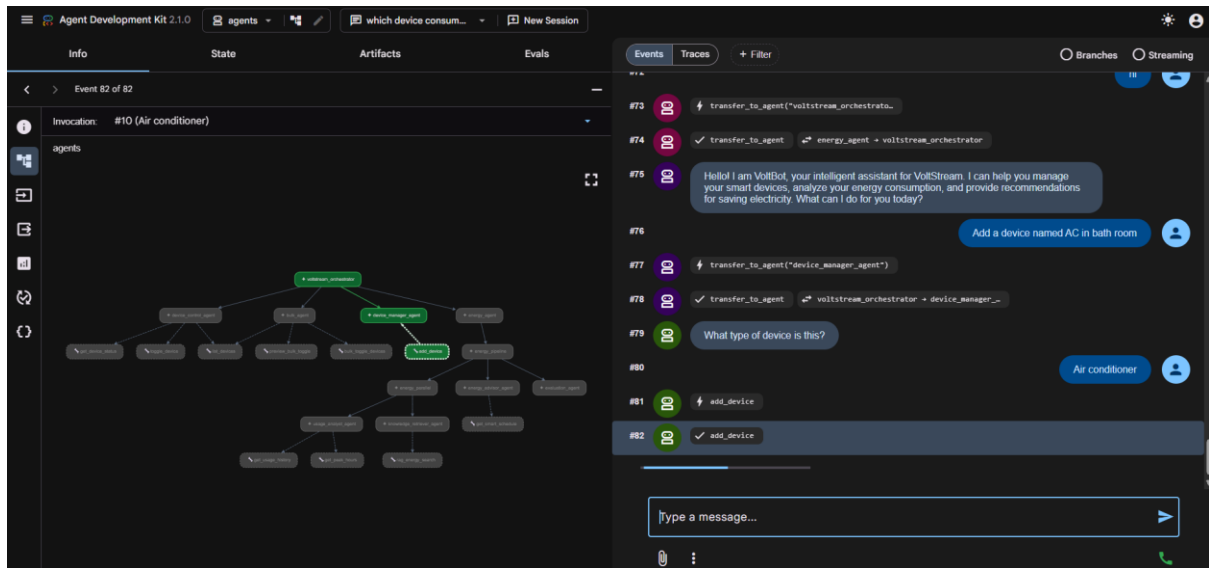


VOLTSTREAM (Week 6)

Intelligent Energy Management Platform RAG Integration and Evaluation

Source Code: <https://github.com/MYethishwar/VoltStream>



Week 6 focused on enhancing the Energy Advisor Agent through the integration of a Retrieval-Augmented Generation (RAG) pipeline. The implementation included PDF ingestion, document chunking, embedding generation, vector storage using ChromaDB, semantic retrieval, and context-aware response generation. An LLM-as-Judge framework was used to evaluate Faithfulness and Relevance, while cloud deployment ensured scalable access to reliable, source-grounded recommendations.

Project Architecture

Week- 6 Multi Agent Architecture with advanced RAG

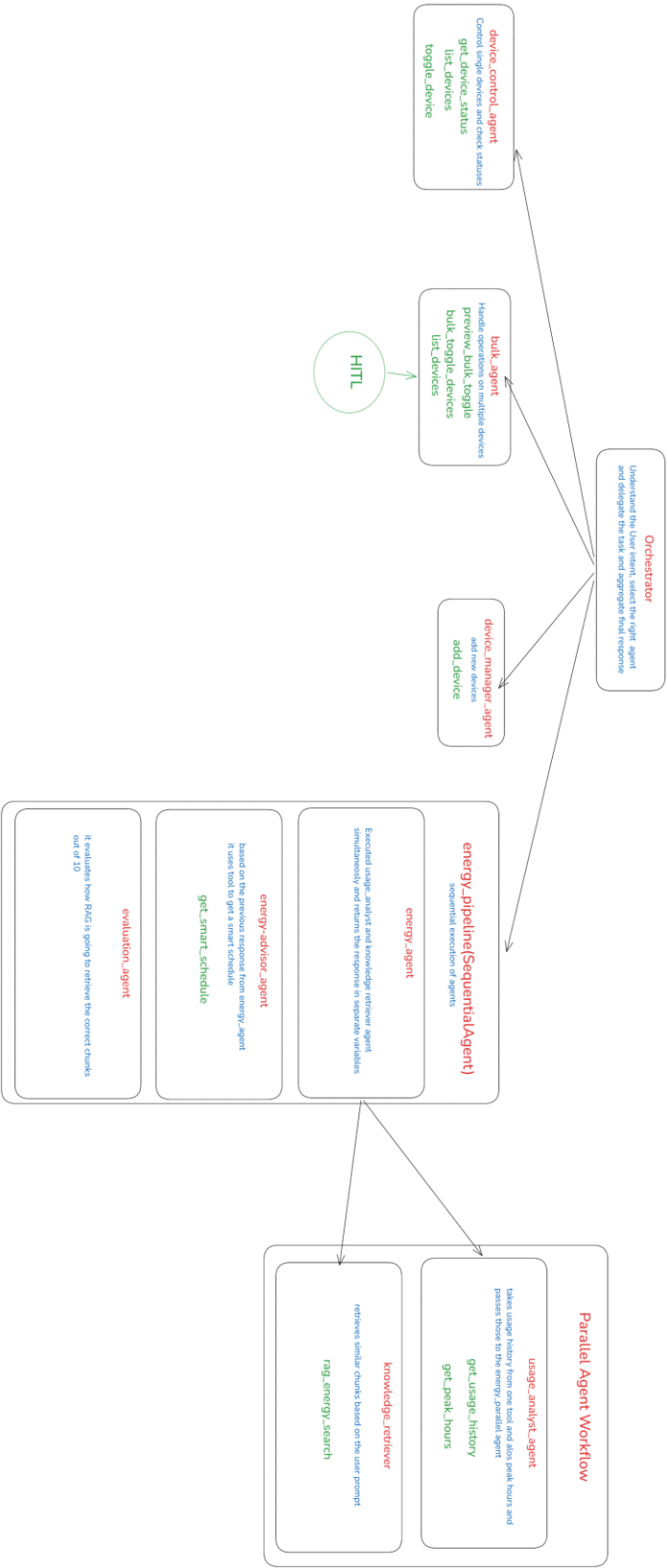


Table of Contents

1. Introduction & Objectives
2. Understanding Project Structure
3. Mult Agent workflow
4. Implementation Details
5. Output & Results

1. Introduction & Objectives

Week 6 focused on enhancing the existing Energy Advisor Agent by integrating Retrieval-Augmented Generation (RAG) capabilities. The primary goal of this enhancement was to improve the reliability and factual accuracy of generated responses by enabling the agent to retrieve relevant information from a document-based knowledge repository before generating an answer. While Large Language Models are highly effective in understanding and generating natural language, they may occasionally produce responses that are not grounded in factual information. To overcome this limitation, a retrieval mechanism was introduced to provide the model with verified contextual information during response generation.

The implementation of RAG represents a significant improvement in the overall intelligence of the agent. Rather than relying solely on pretrained knowledge, the system now has the ability to search, retrieve, and utilize information stored within indexed documents. This approach enables the agent to provide more accurate recommendations, improve transparency through source attribution, and reduce the occurrence of hallucinated responses.

The major objectives of Week 6 included:

- Building a document-based knowledge repository using energy-related PDF documents.
- Processing and indexing documents into a vector database.
- Implementing semantic search capabilities using embeddings.
- Integrating retrieval tools into the Energy Advisor Agent workflow.
- Generating context-aware and source-grounded responses.
- Evaluating response quality using an LLM-as-Judge framework.
- Measuring Faithfulness and Relevance of generated answers.
- Reducing hallucinations through document grounding.
- Deploying the enhanced solution to a cloud environment.
- Establishing a scalable foundation for future knowledge expansion.

The completion of these objectives transformed the Energy Advisor Agent from a purely generative system into a retrieval-enhanced intelligent assistant capable of producing more trustworthy and explainable responses.

2. Understanding Project Structure

The Week 6 enhancement required the interaction of several interconnected components responsible for document processing, vector storage, retrieval, response generation, evaluation, and deployment. Rather than introducing an entirely new architecture, the enhancement leveraged existing modules while extending them to support retrieval-based workflows.

The implementation primarily involved the following components:

2.1 Energy Advisor Agent

The Energy Advisor Agent serves as the primary interface between user queries and the knowledge retrieval system. It is responsible for understanding user requests, determining whether external knowledge is required, invoking retrieval tools when necessary, and generating final responses.

Prior to the enhancement, the agent relied exclusively on language model reasoning. After the integration of RAG, the agent gained access to document-based knowledge and became capable of generating responses grounded in retrieved information.

The responsibilities of the Energy Advisor Agent include:

- Understanding user intent.
- Selecting appropriate tools.
- Requesting knowledge retrieval when required.
- Incorporating retrieved context into prompts.
- Generating grounded recommendations.
- Returning source-aware responses.

2.2 PDF Processing Service

The PDF Processing Service is responsible for converting uploaded documents into a format suitable for retrieval.

Since Large Language Models cannot directly search through PDF files, the contents of the documents must first be extracted and processed. The service performs text extraction and prepares the content for downstream indexing operations.

Key responsibilities include:

- PDF text extraction.
- Content preprocessing.
- Document segmentation.
- Metadata generation.
- Data preparation for indexing.

This component acts as the entry point of the document ingestion pipeline.

2.3 Embedding Service

The Embedding Service converts textual information into numerical vector representations known as embeddings. These vectors capture the semantic meaning of text and enable similarity-based retrieval.

Instead of relying on keyword matching, semantic embeddings allow the system to understand conceptual relationships between queries and documents. This significantly improves retrieval quality and enables relevant information to be found even when exact words do not match.

The embedding service performs:

- Document embedding generation.
- Query embedding generation.
- Semantic representation creation.
- Similarity search preparation.

The all-MiniLM-L6-v2 model was selected because it provides a good balance between retrieval quality and computational efficiency.

2.4 ChromaDB Vector Database

ChromaDB serves as the knowledge storage layer of the RAG system.

After embeddings are generated, they are stored within ChromaDB along with metadata information such as:

- PDF name
- Topic
- Page number
- Chunk identifier
- Source information

The vector database enables efficient similarity searches and allows the retrieval system to identify the most relevant information for a given query.

The database supports:

- Persistent storage.
- Semantic similarity search.
- Metadata filtering.
- Fast retrieval operations.
- Scalable document indexing.

2.5 RAG Service

The RAG Service acts as the central orchestration layer for retrieval operations.

Its responsibilities include:

- Receiving user queries.
- Generating query embeddings.
- Searching ChromaDB.
- Retrieving relevant document chunks.

- Constructing contextual prompts.
- Generating grounded responses.
- Returning source information.

This component forms the bridge between retrieval and generation.

2.6 Evaluation Framework

To assess system performance, an evaluation framework was developed.

The framework measures:

- Faithfulness
- Relevance
- Context utilization
- Response quality
- Source grounding

The evaluation process uses an LLM-as-Judge approach to automatically assess generated answers and determine whether they accurately reflect retrieved information.

3. Multi-Agent Workflow

The Week 6 enhancement introduced a retrieval-driven workflow into the Energy Advisor Agent. The workflow begins when a user submits a query through the application interface.

The first stage involves query analysis. The Energy Advisor Agent receives the user's request and determines whether the query requires information from the knowledge repository. If retrieval is required, the agent invokes the retrieval tool.

Once the retrieval tool is called, the query is forwarded to the RAG service. The service converts the query into an embedding using the embedding model. This embedding represents the semantic meaning of the query and allows the system to perform similarity-based retrieval.

The generated embedding is then compared against document embeddings stored in ChromaDB. Using vector similarity calculations, the database identifies the most relevant document chunks related to the user's request.

After retrieval, the selected chunks are combined into a contextual knowledge block. This context is then injected into the prompt supplied to the language model.

The language model receives:

- User query
- Retrieved document context
- Agent instructions
- Response generation rules

Using this information, the model generates a response that is grounded in retrieved documentation rather than relying solely on pretrained knowledge.

Following response generation, source attribution is performed. The system identifies the primary document source and attaches metadata such as document name and page number.

The final response returned to the user therefore contains:

- Generated answer
- Retrieved context support
- Source information
- Knowledge-base attribution

This workflow significantly improves transparency and enables users to understand where the generated information originated.

For more complex scenarios, multiple components may operate together. Retrieval, evaluation, and response generation can function as coordinated stages within the workflow. This modular design allows additional capabilities to be integrated in future iterations without major architectural modifications.

4. Implementation Details

The implementation phase involved multiple stages that collectively formed the Retrieval-Augmented Generation pipeline.

4.1 Document Ingestion

The first step involved collecting and processing energy-related PDF documents.

These documents contained:

- Energy efficiency guidelines.
- HVAC recommendations.
- Appliance usage information.
- Renewable energy resources.
- Government energy standards.

The uploaded documents served as the knowledge source for retrieval operations.

4.2 Text Extraction

Once uploaded, the documents were processed to extract textual content.

The extraction process involved:

- Reading document pages.
- Extracting textual information.
- Preserving document structure.
- Removing unnecessary formatting artifacts.

The extracted text formed the basis for indexing and retrieval.

4.3 Text Chunking

Large documents cannot be stored efficiently as a single unit.

To address this challenge, extracted content was divided into smaller chunks.

The chunking strategy used:

- Fixed chunk sizes.
- Overlapping segments.
- Context preservation.
- Improved retrieval granularity.

The use of overlapping chunks ensured that information spanning multiple sections remained accessible during retrieval.

Benefits of chunking included:

- Better retrieval precision.
- Reduced context loss.

- Improved semantic matching.
- Efficient storage.

4.4 Embedding Generation

After chunking, each chunk was transformed into an embedding.

Embeddings represent text in a high-dimensional vector space where semantically similar information is located closer together.

The embedding process enabled:

- Semantic understanding.
- Similarity comparison.
- Context matching.
- Efficient retrieval.

This stage is fundamental to modern retrieval systems because it allows the system to search by meaning rather than exact wording.

4.5 Vector Storage

Generated embeddings were stored within ChromaDB.

Alongside embeddings, metadata was also preserved.

Metadata included:

- PDF name
- Topic
- Page number
- Chunk identifier
- Source reference

This metadata later enabled source attribution and traceability.

4.6 Semantic Retrieval

When a query is received, the same embedding process is applied to the query.

The retrieval pipeline then:

1. Generates a query embedding.
2. Searches ChromaDB.
3. Retrieves relevant chunks.
4. Ranks results by similarity.
5. Selects top relevant chunks.

This process ensures that the most useful information is supplied to the language model.

4.7 Context Injection

Retrieved document chunks are combined into a structured context block.

The context is inserted into the language model prompt before response generation.

This process enables the model to:

- Access retrieved information.
- Use document knowledge.
- Generate grounded responses.
- Reduce unsupported claims.

Context injection is one of the most important stages in the RAG pipeline because it directly influences response quality.

4.8 Response Generation

After context injection, the language model generates the final answer.

The response generation stage uses:

- User question
- Retrieved context
- Agent instructions
- Response guidelines

The generated answer is expected to be:

- Accurate
- Concise
- Context-aware
- Source-grounded

4.9 Evaluation Framework

To measure system effectiveness, an LLM-as-Judge evaluation framework was implemented.

The evaluation focused on two primary metrics:

Faithfulness

Faithfulness measures how accurately the generated answer reflects the retrieved context.

A highly faithful answer contains information directly supported by retrieved documents.

Relevance

Relevance measures how effectively the answer addresses the user's question.

A highly relevant answer directly satisfies the user's information needs.

The evaluation process involved:

- Creating test questions.
- Generating responses.

- Scoring outputs.
- Identifying failure cases.
- Measuring overall performance.

4.10 Deployment

The final stage involved deployment.

The application was containerized using Docker and deployed to Google Cloud Run.

Deployment activities included:

- Container creation.
- Dependency management.
- Environment configuration.
- Cloud deployment.
- Endpoint testing.
- Performance validation.

The deployment process ensured that the enhanced agent could be accessed reliably in a cloud environment.

5. Output & Results

The Week 6 enhancement successfully achieved all major objectives related to Retrieval-Augmented Generation integration.

The document indexing pipeline was successfully implemented, allowing multiple PDF documents to be processed and stored within the vector database. Semantic retrieval functionality enabled relevant information to be identified efficiently and supplied to the language model during response generation.

The integration of retrieval tools significantly improved the quality of generated responses. Answers became more grounded in document content and demonstrated improved factual consistency. The inclusion of source attribution further increased transparency by allowing users to identify the origin of retrieved information.

The evaluation framework provided measurable insights into system performance. Most test cases achieved satisfactory Faithfulness and Relevance scores, indicating that the retrieved context was being effectively utilized during answer generation. The evaluation process also helped identify limitations and areas requiring future improvement.

The deployment phase confirmed that the enhanced solution could operate successfully within a cloud environment. Retrieval operations, response generation, and evaluation workflows functioned correctly under deployment conditions.

Key outcomes achieved during Week 6 include:

- Successful integration of Retrieval-Augmented Generation.
- Creation of a searchable document knowledge repository.
- Implementation of semantic retrieval using embeddings.
- Integration of retrieval tools with the Energy Advisor Agent.
- Source-aware response generation.
- Evaluation using Faithfulness and Relevance metrics.
- Reduced hallucination risk.
- Successful cloud deployment.
- Establishment of a scalable foundation for future enhancements.

Overall, Week 6 demonstrated the effectiveness of combining retrieval systems with language models to produce more reliable, transparent, and context-aware responses. The implemented architecture provides a strong foundation for future developments involving larger knowledge repositories, advanced retrieval techniques, and enhanced multi-agent collaboration.